

HANDWRITTEN DIGIT RECOGNITION SYSTEM USING PYTHON

Submitted By

Name: Tanishka Jain

Course: B.Tech Computer Science Engineering

Internship: Pinnacle Grid Internship

Task: Task-4 – Handwritten Digit Recognition System Using Python

Introduction

Artificial Intelligence (AI) and Machine Learning (ML) are transforming the way computers solve real-world problems. One of the most popular applications of machine learning is image recognition, where a computer identifies objects or patterns from images. Handwritten Digit Recognition is one of the fundamental problems in image classification and is widely used to understand the concepts of deep learning and computer vision.

In this project, a Handwritten Digit Recognition System was developed using Python and TensorFlow. The system is trained on the MNIST dataset, which contains thousands of handwritten digit images ranging from 0 to 9. The trained model learns the patterns of handwritten digits and predicts the correct digit for new unseen images.

This project demonstrates the complete machine learning workflow including dataset loading, preprocessing, model creation, training, testing, evaluation, and prediction.

Objective

The primary objective of this project is to build a machine learning model capable of recognizing handwritten digits accurately.

The objectives are:

- To understand image classification.
 - To use the MNIST dataset.
 - To preprocess image data.
 - To train a Neural Network model.
 - To evaluate model performance.
 - To predict handwritten digits.
 - To understand TensorFlow and Keras libraries.
-

Problem Statement

Recognizing handwritten digits manually is time-consuming and may result in errors. Banks, schools, postal departments, and examination systems process thousands of handwritten documents daily.

The objective of this project is to automate handwritten digit recognition using Machine Learning. The model should accurately classify handwritten digits from 0 to 9.

Dataset Description

The MNIST dataset is one of the most famous datasets used in Machine Learning.

Dataset Information

- Dataset Name: MNIST
- Total Images: 70,000
- Training Images: 60,000
- Testing Images: 10,000
- Image Size: 28×28 pixels
- Color: Grayscale

- Classes: 10 (Digits 0–9)

Each image represents a handwritten digit written by different people.

Technologies Used

The following technologies were used in this project:

- Python
 - TensorFlow
 - Keras
 - NumPy
 - Matplotlib
 - VS Code
 - GitHub
-

Software Requirements

- Windows 10/11
 - Python 3.12
 - VS Code
 - TensorFlow
 - NumPy
 - Matplotlib
 - Internet Connection
 - GitHub Account
-

Hardware Requirements

- Intel Core i3/i5/i7 Processor
- Minimum 4GB RAM

- 2GB Free Storage
 - Windows Operating System
-

Python Libraries Used

TensorFlow

TensorFlow is an open-source deep learning framework developed by Google. It is used to create and train neural network models.

Keras

Keras is a high-level API of TensorFlow that simplifies deep learning model development.

NumPy

NumPy is used for numerical operations and handling arrays efficiently.

Matplotlib

Matplotlib is used to visualize graphs and display prediction images.

Project Workflow

The project follows these steps:

1. Load MNIST dataset.
 2. Normalize image pixels.
 3. Create Neural Network model.
 4. Compile the model.
 5. Train the model.
 6. Evaluate the model.
 7. Save the trained model.
 8. Predict handwritten digits.
 9. Display prediction results.
-

Data Preprocessing

Before training, the dataset was preprocessed.

The preprocessing steps include:

- Loading the MNIST dataset.
- Dividing pixel values by 255.
- Converting pixel values between 0 and 1.
- Separating training and testing data.

Data preprocessing improves model accuracy and training performance.

Model Architecture

A Sequential Neural Network was used.

The architecture consists of:

- Flatten Layer
- Dense Layer (128 neurons, ReLU)
- Dense Layer (64 neurons, ReLU)
- Output Layer (10 neurons, Softmax)

The Flatten layer converts the 28×28 image into a one-dimensional vector.

The ReLU activation function introduces non-linearity, while the Softmax activation function predicts the probability of each digit.

Model Compilation

The model was compiled using:

Optimizer

Adam Optimizer

Loss Function

Sparse Categorical Crossentropy

Evaluation Metric

Accuracy

The Adam optimizer provides faster convergence and better performance.

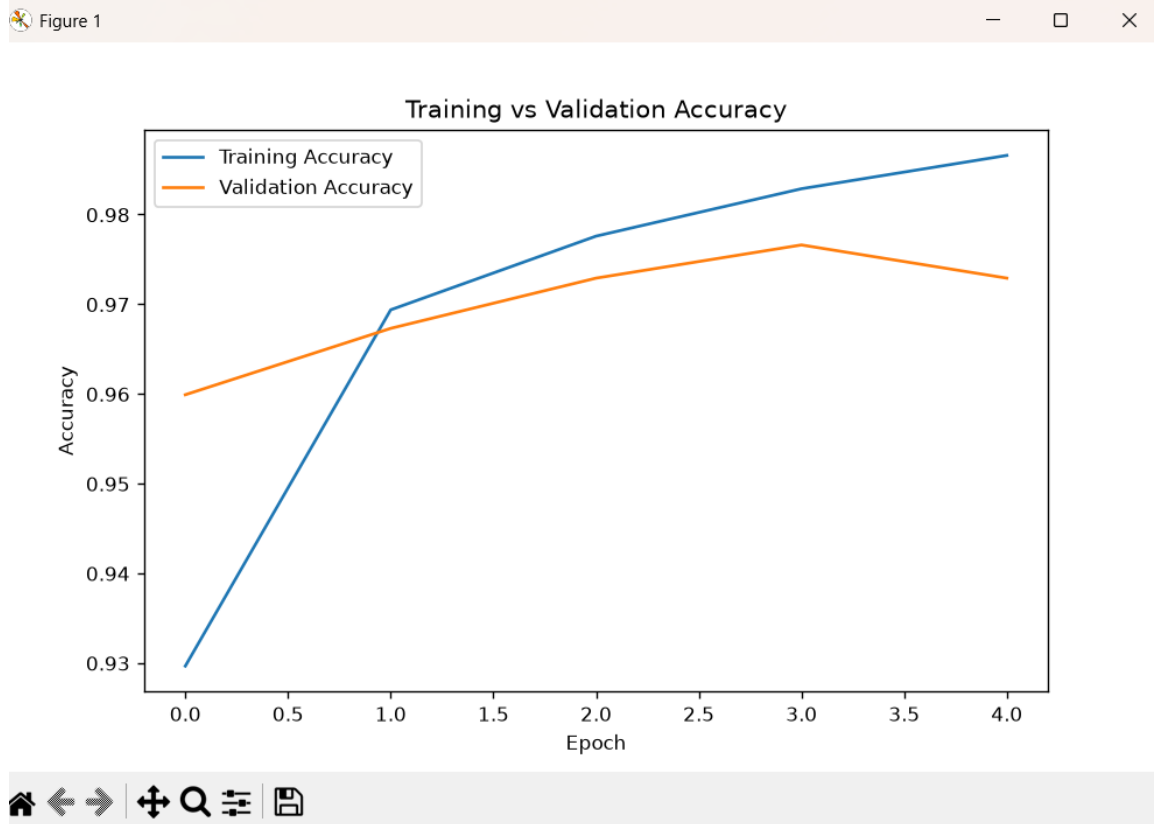
Model Training

The model was trained using:

- Epochs: 5
- Training Images: 60,000
- Validation Images: 10,000

During training, the model continuously improved its prediction capability by minimizing the loss function.

The training accuracy increased with every epoch.



Model Evaluation

After training, the model was tested using unseen images.

The testing dataset contains 10,000 handwritten digit images.

The model successfully recognized most handwritten digits with very high accuracy.

```
PROBLEMS 9 OUTPUT DEBUG CONSOLE TERMINAL PORTS python3.12 + - - -
PS C:\Users\Anuj Kumar Jain\OneDrive\Desktop\Task-4-Digit-Recognition> python train_model.py
l not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/5
1875/1875 11s 5ms/step - accuracy: 0.9297 - loss: 0.2379 - val_accuracy: 0.9599 - val_loss: 0.1256
Epoch 2/5
1875/1875 10s 5ms/step - accuracy: 0.9694 - loss: 0.1000 - val_accuracy: 0.9673 - val_loss: 0.1042
Epoch 3/5
1875/1875 9s 5ms/step - accuracy: 0.9776 - loss: 0.0702 - val_accuracy: 0.9729 - val_loss: 0.0850
Epoch 4/5
1875/1875 9s 5ms/step - accuracy: 0.9829 - loss: 0.0548 - val_accuracy: 0.9766 - val_loss: 0.0786
Epoch 5/5
1875/1875 10s 5ms/step - accuracy: 0.9866 - loss: 0.0414 - val_accuracy: 0.9729 - val_loss: 0.0906
313/313 1s 3ms/step - accuracy: 0.9729 - loss: 0.0906

Test Accuracy: 0.9728999733924866
```


Results

The developed model achieved excellent performance.

Training Accuracy

Approximately **98%**

Validation Accuracy

Approximately **97%**

The prediction output correctly identified handwritten digits.

Example:

Actual Digit = 0

Predicted Digit = 0

This indicates that the model learned the digit patterns effectively.

Advantages

- High prediction accuracy.
 - Fast digit recognition.
 - Easy to implement.
 - Uses open-source libraries.
 - Suitable for beginners.
 - Can be extended for real-world applications.
-

Limitations

- Works only on handwritten digits.
 - Performance depends on image quality.
 - Cannot recognize alphabets.
 - Requires sufficient training data.
-

Applications

Handwritten Digit Recognition has numerous practical applications:

- Bank cheque processing
 - Postal ZIP code recognition
 - Automatic form processing
 - Educational applications
 - Examination systems
 - OCR (Optical Character Recognition)
 - Banking Automation
 - Healthcare forms
 - Government records
 - Data entry automation
-

Future Scope

This project can be further enhanced in several ways:

- Implement Convolutional Neural Networks (CNNs) for higher accuracy.
 - Develop a real-time handwritten digit recognition application.
 - Create a web application using Flask or Streamlit.
 - Deploy the model on cloud platforms.
 - Add support for handwritten alphabets and symbols.
 - Improve prediction speed.
 - Use larger datasets for training.
 - Integrate with mobile applications.
-

Conclusion

The Handwritten Digit Recognition System was successfully developed using Python, TensorFlow, and the MNIST dataset. The project demonstrated the complete machine learning pipeline, including data preprocessing, model building, training, evaluation, and prediction.

The neural network achieved approximately **97–98% accuracy**, showing that deep learning models are highly effective for handwritten digit classification. The project enhanced understanding of machine learning concepts, neural networks, image processing, and TensorFlow. It also provided practical experience in building an end-to-end AI application.

Overall, this project serves as an excellent foundation for learning deep learning and image classification techniques and can be extended to more advanced computer vision applications in the future.

References

1. TensorFlow Official Documentation
2. Keras Documentation
3. MNIST Handwritten Digit Dataset
4. Python Official Documentation
5. NumPy Documentation
6. Matplotlib Documentation
7. Scikit-learn Documentation
8. Deep Learning by Ian Goodfellow
9. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow
10. Machine Learning research articles and online tutorials